

Facial Reconstruction Atlas — Technical Overview

Prepared for: UVA Health Information Technology

Prepared by: Ayaan Siddiqui, Research Assistant, Department of Otolaryngology

Principal Investigator: Dr. Samuel Oyer

IRB Protocol: 18181

Date: 5 September 2026

Document status: For review

Application version: 0.4.0 (prototype)

Prepared in response to the Technical Documentation action item from the 27 August 2026 meeting. It covers the application's software and third-party packages, code size and deployment structure, database and image-path usage, the data elements displayed and their consent constraints, and the operational characteristics needed to evaluate hosting on UVA infrastructure.

Section 11 lists the items still needed from Health IT, and Section 10 records the development work assigned to this project team.

1. Purpose and scope

The Facial Reconstruction Atlas is an internal educational reference application. It lets clinicians and trainees search a series of facial plastic surgery reconstructive cases by anatomic defect location, repair method, flap type, and graft characteristics, and review the staged photographic series for each matching case.

It is a **read-oriented query tool**, not an interactive planning or reconstruction system. Users filter to find comparable prior cases and review the associated treatment course and outcome.

The case series was collected by the Department of Otolaryngology under **IRB protocol 18181**. The application makes that existing series searchable, replacing the current workflow of scanning a spreadsheet by hand and locating the corresponding photograph folders on the O drive.

1.1 Scope boundary

The application is scoped to **internal UVA Health use only**.

An earlier concept included sharing the atlas with surgeons and residents outside UVA. That scope was withdrawn before development proceeded, following a review of the governing IRB protocol and the patient consent forms, which indicated that the consent obtained does not extend to distributing patient photographs outside the institution.

Any future expansion beyond internal use would require IRB amendment and, in all likelihood, re-consent. No such expansion is proposed here. The team recognizes that external access would additionally require a more involved security, privacy, and operational review.

2. Current status

The application is a **working prototype**. Functionality and user interface are substantially complete; the principal remaining implementation work is connecting the application to the selected patient-image files on the O drive, which depends on the hosting and share-access decisions under evaluation.

No protected health information has been loaded into the application at any point. All images currently in the system are generated placeholder graphics. See Section 8.3.

The prototype's security model was deliberately left minimal rather than built speculatively, because the institutional requirements — authentication method, access control model, audit obligations, and supported database platform — were not known until the 27 August meeting. Since that meeting, the controls that did not depend on those answers have been closed; Section 10.3 lists them, Section 10.4 lists what remains open and why, and Section 10.2 records the development work still to come.

3. Software stack and third-party packages

3.1 Runtime

LAYER	TECHNOLOGY	VERSION
Language	Python	≥ 3.12
Web framework	FastAPI	≥ 0.110.0
ASGI server	Uvicorn (<code>uvicorn[standard]</code>)	≥ 0.27.0
Database	SQLite	Python standard library
Frontend	Vanilla JavaScript, HTML, CSS	No framework

3.2 Direct third-party dependencies

The complete dependency list is four packages:

PACKAGE	PURPOSE	REQUIRED AT
<code>fastapi</code>	HTTP framework, routing, request validation	Runtime
<code>uvicorn[standard]</code>	ASGI server	Runtime
<code>openpyxl</code>	Reads the source spreadsheet	Runtime — see below
<code>pillow</code>	Generates the sample placeholder images	Build / development only

`pillow` is used only by the sample-image generator, which the build step invokes as a separate process. The application itself never imports it, so it is not required on a production host once images are in place.

`openpyxl` is required at runtime as the code currently stands, which is worth stating precisely because it is not obvious. The application imports the `metadata-import` module at startup, and that module reads the source spreadsheet at import time — so the spreadsheet must be present on the application host, and `openpyxl` installed, for the process to start. The data itself is only needed to seed an empty database.

This is a known wart rather than a design intent, and it is a small change to fix: deferring the spreadsheet read until the seeding routine actually calls for it makes both `openpyxl` and the spreadsheet optional on a host whose database is already populated. It is on the work list, and this document will be updated when it lands. Flagged here because it affects two things Health IT will care about — what must be installed on the application host, and where the source spreadsheet has to live in production.

FastAPI pulls in Pydantic and Starlette transitively; `uvicorn[standard]` pulls in `httptools`, `uvloop`, `websockets`, and `watchfiles`.

There are no JavaScript package dependencies and no frontend build step. The browser loads three files directly. There are no CDN references, no external fonts, and no third-party analytics or tracking.

There are no outbound network calls at runtime. The application does not contact any external service.

3.3 Supply chain note

Four direct dependencies, all widely used and actively maintained, with no vendored or unmaintained code. Version constraints are currently lower bounds (`>=`); these can be pinned to exact versions with a lockfile if that is required for the deployment review.

4. Code size and repository structure

Approximately **6,700 lines** across ten files, plus a 270-line README.

FILE	LINES	ROLE
<code>styles.css</code>	1,456	Frontend styling
<code>app.py</code>	1,233	FastAPI application: all routes, auth, database access, media serving
<code>script.js</code>	1,209	Frontend application logic
<code>region-editor.html</code>	885	Development tool — anatomy diagram region editor
<code>import_patient_log.py</code>	614	Spreadsheet → database import
<code>index.html</code>	577	Single-page application shell
<code>generate_placeholders.py</code>	324	Development tool — sample image generator
<code>validate_spreadsheet.py</code>	235	Pre-flight spreadsheet validation
<code>field_options.py</code>	115	Filter vocabularies and column definitions
<code>build.py</code>	30	Build step: stages sample data and seeds the database

Grouped by role:

- **Backend, production:** ~2,200 lines (`app.py` , `import_patient_log.py` , `validate_spreadsheet.py` , `field_options.py`)
- **Frontend:** ~3,240 lines (`index.html` , `script.js` , `styles.css`)
- **Development and build tooling:** ~1,240 lines, excludable from a production deployment (`region-editor.html` , `generate_placeholders.py` , `build.py`)

The repository is flat — no packages, no submodules, no monorepo structure.

4.1 Repository for evaluation and proof of concept

github.com/ayaanrsiddiqui/facial-reconstruction-atlas

This repository contains the complete application and **a fabricated twelve-case sample spreadsheet**. The build step generates the placeholder images and seeds the database from it, so a fresh clone runs standalone with no configuration and no data files to supply. Version control excludes every real case log, database, and image directory by rule — the repository contains **no real research records of any kind**, and none can be committed to it accidentally.

That makes it the appropriate target for the proof-of-concept deployment: the server team can stand the application up, exercise it end to end, and confirm the hosting model works without any real data being involved in a first deployment attempt on unfamiliar infrastructure.

The working repository holding the real 143-case metadata is separate and is not needed for the hosting evaluation. Loading real data is a configuration step once a deployment target is confirmed, and is gated behind the approval item in Section 11.

The team is happy to change the repository's visibility, transfer it to a UVA-owned organization, or mirror it internally — see question 15 in the accompanying email.

5. Architecture and request flow

5.1 Components

A single application server process, a metadata database, and a directory of image files. No message queues, no caches, no background workers, no external services.

5.2 Request flow

All access to case data and images passes through the application server. **The image store is never exposed directly to the browser** — there are no direct file URLs, no shared links, and no client-side storage credentials.

```
1. Authenticated browser →HTTPS→ Application server
2.                               Application server → Database (case metadata)
3.                               Application server ← case data + image filenames
4.                               Application server → Image share (read bytes)
5.                               Application server ← image bytes
6. Authenticated browser ←HTTPS← JSON case data + streamed images
```

This was a deliberate design choice: every access to patient media traverses one point in the application. That is what makes per-user authorization and access logging implementable as a single instrumentation layer rather than a re-architecture, and it is relevant to the logging requirement in Section 11.

5.3 Endpoint inventory

Twenty-one routes — sixteen API endpoints and five static-file routes. Full inventory in **Appendix B**.

6. Database

6.1 Current implementation

SQLite, accessed through the Python standard library. Seven tables in two groups:

Case data (rebuilt from the source spreadsheet on import)

- `patients` — one row per case; 12 columns
- `patient_locations` — one row per defect site per case; a case may have several
- `images` — one row per photograph; filename, reconstruction stage, sort order

Application data (generated in use)

- `users` — local accounts, expected to be removed when SSO is in place
- `sessions` — active session tokens
- `favorites` — per-user saved cases
- `comments` — per-user case annotations

Full schema with column types and constraints in **Appendix A**.

The database file is presently about 150 KB with the 143-case metadata set loaded. It is not a significant storage consideration at any anticipated scale.

6.2 SQL Server migration assessment

Per the 27 August discussion, Microsoft SQL Server is the supported platform, with SQL Server Express suggested for development.

The schema is simple and portable: seven tables, integer primary keys, standard foreign keys, no stored procedures, no triggers, no views, no database-specific extensions. All SQL in the application is parameterized and uses common syntax. The changes required are tractable:

ITEM	CHANGE REQUIRED
INTEGER PRIMARY KEY AUTOINCREMENT	→ <code>INT IDENTITY(1,1) PRIMARY KEY</code>
<code>TEXT</code> columns	→ <code>NVARCHAR(n)</code> / <code>NVARCHAR(MAX)</code>
<code>datetime('now')</code> defaults	→ <code>SYSUTCDATETIME()</code>
<code>COLLATE NOCASE</code> on <code>users.username</code>	→ a case-insensitive SQL Server collation
<code>sqlite3</code> driver calls	→ <code>pyodbc</code> (adds a dependency and an ODBC driver)
Connection handling	Connection pooling, and credential management for the DB account

The most substantive change is the driver layer, which is currently `sqlite3` calls written inline rather than behind an abstraction. **Estimated effort: one to two days**, most of it in mechanical replacement and re-testing rather than design.

This project team can make the change. It would be useful to know whether SQL Server is required for the initial deployment or whether SQLite is acceptable for a proof of concept, since that affects sequencing against the SAML work.

7. Image storage and path handling

7.1 The image set

Per the 27 August meeting: approximately **4.2 GB**, stored on the **O drive** under the project team's folder structure, in a dedicated folder containing one subfolder per patient number, with roughly **4 to 15 images per patient**.

These are copies placed in a separate folder for this project; the original patient-photo materials remain in their source location, so moving or copying the selected set does not affect the source data.

The image count per case is relevant to the remaining implementation work. The prototype was built against a generated placeholder set using a fixed six-stage naming convention; the real folders hold a variable number of files under the department's own naming. The import must therefore be changed to enumerate the actual directory rather than derive filenames from the spreadsheet. This is item 1 in Section 10.2.

William is providing the precise path and the location of the related metadata. A sample directory listing from two or three case folders has also been requested, to establish the actual file naming convention — see Section 10.2, item 1.

7.2 How the application resolves an image path

The database stores a **folder name and a filename** — not absolute paths. A single root constant is joined to those two values:

```
<image store root> / <folder_name> / <filename>
```

`folder_name` is the per-case folder (a patient number); `filename` is the individual photograph. This means **relocating the image store changes one configuration value**, not any stored data.

The root is read from `ENTDATABASE_IMAGE_ROOT`, and the database location from `ENTDATABASE_DB_PATH`, so both can be pointed at their production locations without a code change. The database contents do not change when the image store moves.

7.3 Path safety

Image requests are constrained to the image store by three independent checks:

1. Path segments containing `/`, `\`, or `..` are rejected outright.
2. The resolved absolute path is verified to sit beneath the image store root, so symlinks and encoded traversal cannot escape it.
3. Only files bearing an extension from a fixed allowlist are served (`.jpg`, `.jpeg`, `.png`, `.gif`, `.webp`, `.bmp`).

7.4 Share access — open question

The application reads image bytes from the share on behalf of the requesting user. It does not currently implement any share-level authentication, because the access model has not been determined. Whether the hosted application should use a service account, impersonate the requesting user, or use another controlled method is item 3 in Section 11.

If a service account is used, the application will hold credentials to the share and becomes the sole enforcement point for who may view which images — which makes the application-level access control and logging in Section 10 load-bearing rather than supplementary. This project team would note that as a factor in the decision.

8. Data elements, PHI, and consent constraints

8.1 What is displayed

Each case record displays:

FIELD	EXAMPLE	IDENTIFIER?
Study identifier	ENT - 042	Study number, not an MRN
Defect location	Region and sub-location	No
Defect size	Measurement	No
Full-thickness indicator	Yes / No	No
Method of repair	Free text from a controlled vocabulary	No
Flap type and description	Free text	No
Graft type and donor site	Free text	No
Surgical notes	Free text ("surgical pearls")	No
Photographic series	4–15 staged images	Yes — facial photographs

Not displayed anywhere in the application: patient name, MRN, age, sex, dates of service, or any other demographic field. These are not present in the application's database at all.

8.2 PHI classification

DATA	CLASSIFICATION	CURRENTLY IN SYSTEM
Case metadata	De-identified research data	Yes — 143 records
Patient photographs	PHI	No — placeholders only
User accounts, favorites, comments	Internal application data	Yes

Facial photographs are identifiable by nature and are treated as PHI regardless of the de-identification applied to the accompanying metadata. This is the constraint that drove the internal-only scope and that governs the requirements in Sections 10 and 11.

8.3 Current image data

The application currently serves **generated placeholder graphics** — simple face outlines with the case's defect region highlighted — so that the interface can be demonstrated and reviewed without any patient image leaving the departmental share. No patient photograph has been copied into the application, its repository, or any deployment.

8.4 The identifier key

A **separate key stored on the O drive links study identifiers to identifying information.** That key is not part of this application, is not accessible to it, and is not proposed to be. Its existence means the case records are re-identifiable by anyone holding both the application data and the key, which is why the case metadata is described here as de-identified for application purposes rather than as a de-identified data set in the regulatory sense.

The team understands from the 27 August discussion that the consent obtained and the IRB-approved scope are central to the privacy review, and that the exact data elements and storage relationships need to be documented. Section 8.1 and this subsection are intended to satisfy that; the team is glad to expand either.

8.5 Import pipeline

Case metadata is loaded from the department's spreadsheet by `import_patient_log.py`, a read-only transform. Relevant properties for a data-integrity review:

- Columns are matched by **header text**, not position, tolerating whitespace and capitalization differences.
- Ten columns are required; if any is missing the import **fails immediately** naming the missing columns rather than importing partial data.
- Free-text defect locations are parsed into structured region and sub-location pairs. Unrecognized values are imported and remain keyword-searchable, flagged with a warning rather than silently dropped.
- Rows the department marked incomplete are imported and **visibly flagged in the interface** rather than presented as complete.
- Duplicate study identifiers are dropped with a warning; the first occurrence wins.

`validate_spreadsheet.py` performs a read-only pre-flight check on a candidate spreadsheet and reports dropped rows, unrecognized vocabulary, and mismatches against the image store before anything is imported.

9. User population and expected load

CHARACTERISTIC	VALUE
Expected users at launch	~20
Expected ceiling	< 50
Audience	UVA Health residents, medical students, and internal learners
Usage pattern	Read-only search and image viewing; very low write volume
Concurrency	Expected low single digits
Metadata volume	143 cases; ~156 KB database
Image volume	~4.2 GB, growing slowly as cases are added

Writes are limited to favorites and comments — a handful of small rows per user. The workload is overwhelmingly image reads. This is a small application by any infrastructure measure, and resource requirements should be modest.

One item worth flagging for the access-control discussion: the audience is primarily UVA Health personnel, but **undergraduate students holding general University of Virginia accounts may require separate consideration**, since they would authenticate successfully against the University identity provider without being part of the intended clinical audience. This is an argument for scoping access to a named security group rather than to "any authenticated user."

10. Security controls and planned work

10.1 Implemented today

Media path validation — three independent checks constrain image requests to the image store (Section 7.3). Directory traversal is the primary attack against a file-proxying endpoint and is defended in depth.

Credential storage — local passwords are stored as PBKDF2-HMAC-SHA256 derivations, 200,000 iterations, with a unique 16-byte random salt per user. Verification uses constant-time comparison. Plaintext passwords are never stored or logged.

Injection defenses — all SQL is parameterized; no user-supplied value is interpolated into a query string. The only dynamic SQL fragments are column names drawn from a fixed internal allowlist. User-generated comment text is HTML-escaped before rendering.

Session handling — 256-bit random tokens delivered in an `HttpOnly`, `SameSite=Lax` cookie, unreadable by JavaScript and not sent on cross-site requests.

Object-level authorization — comment deletion verifies that the requesting user owns the comment, rather than relying on the interface not offering the option.

10.2 Assigned development work

These are the items this project team took as action items on 27 August, plus one identified since, in the order they will be addressed.

1. Image enumeration in the import pipeline. The prototype derives image filenames from stage columns in the spreadsheet, producing a fixed six-file naming convention that matches the generated placeholder set. The real case folders hold a variable number of photographs under the department's own naming. The import will be changed to enumerate each case directory and record the filenames actually present, deriving stage from naming convention or file order. This is the substance of "connecting the application to the selected image files" and is the largest remaining implementation item. It is unblocked by a sample directory listing rather than by the full share.

2. SAML authentication against Entra ID. Replacing local accounts with redirect-based authentication to UVA Health Microsoft authentication, so that Entra ID handles credentials and multifactor authentication and the application never receives a user password. This is the first development priority. Coordination with Ronald's team is needed on UVA-specific configuration and testing.

3. Group-scoped access control. Restricting authentication to an approved security group rather than admitting every successfully authenticated person — the authentication-versus-authorization distinction raised in the meeting, and relevant to the undergraduate-account point in Section 9.

4. Normal-user and administrator roles. The application currently has a single user type and no separate administrative role. Planned split: a normal user who searches and views, and a privileged administrator who adds or edits images and database records.

5. SQL Server migration. Per Section 6.2, pending confirmation of whether it is required for the initial deployment.

6. Application access logging. Per the meeting, the most important initial audit information is who accessed the application. Because all media access already traverses a single endpoint, this is one instrumentation layer. Requirements are needed before implementation — Section 11.

10.3 Interim controls — closed since the meeting

The following were open at the time of the 27 August discussion and have since been addressed. They required nothing from Health IT.

Authentication now covers every data endpoint. At the meeting, authentication gated only favoriting and commenting; case search, case detail, filter vocabularies, comments, and image retrieval were all reachable without a session. All six now require an authenticated user. The rule is authenticated-by-default, with no exception list.

Cross-origin policy is now an explicit allowlist. The permissive configuration has been replaced with an origin allowlist read from `ENTDATABASE_ALLOWED_ORIGINS`, with methods narrowed to `GET`, `POST`, `DELETE` and headers to `Content-Type`.

Development routes are disabled by default. The region editor and its read/write endpoints — one of which rewrote a file on the server — now return 404 unless `ENTDATABASE_DEV_TOOLS=1` is set, and additionally require an authenticated user when it is. They are excluded from a production deployment by configuration rather than by convention.

Image and database locations are configuration values. Both are read from the environment (`ENTDATABASE_IMAGE_ROOT`, `ENTDATABASE_DB_PATH`) rather than being fixed in source, so the application can be pointed at the departmental share and at a hosted database without code changes. The image root is resolved once at startup, which is what the path-containment check in Section 7.3 compares against.

10.4 Interim controls still open

Account registration is open to any party who can reach the application. This will be closed in the interim and removed entirely once SSO is in place.

Local password policy is minimal (6-character minimum), set on the basis that the demonstration contains no real data. SSO removes the need to manage normal-user passwords locally; any remaining administrative or service credentials will require appropriate controls.

Sessions are not expired server-side. A token remains valid until explicit logout. Server-enforced expiry is planned, and is one of the few items here that survives the move to SSO — a local session is still issued after the identity provider's assertion is validated.

Authentication endpoints are not rate-limited. Largely moot once SSO replaces local authentication.

The first, second and fourth of these are all attached to local password authentication, which SSO removes. Rather than invest in code scheduled for deletion, the intent is to close registration in the interim and let the SAML work retire the rest.

11. Items needed from Health IT

1. **Hosting platform and serving model.** Whether the FastAPI application can run on existing UVA/IIS infrastructure — for example IIS with Uvicorn behind it — and whether a proof-of-concept deployment should precede full migration. *(Ronald)*
2. **SAML / Entra ID integration details.** Identity provider metadata endpoint, the attribute carrying user identity, whether group membership is exposed as a claim, and the service-provider registration process. *(Ronald)*

3. **Image share access method.** Whether the hosted application should read the O drive share using a service account, the requesting user's identity, or another controlled method — with the note in Section 7.4 about what each implies for application-level enforcement. *(Ronald)*
 4. **Security group.** Which group should gate access, and who administers membership.
 5. **Logging requirements.** Minimum fields, retention period, and whether records must be forwarded to existing security monitoring. *(Cory, Ronald)*
 6. **Database requirement.** Whether SQL Server is required for initial deployment or whether SQLite is acceptable for a proof of concept.
 7. **Administrative governance.** Who approves users, maintains the application, updates the database, and manages images — and how privileged credentials should be documented and managed. *(Cory)*
 8. **Change management.** The applicable change-management and software-development lifecycle expectations once the hosting approach is settled. *(Cory)*
 9. **Approval gate.** The review or approval that must be satisfied before patient photographs may be loaded into the application.
-

12. Current deployment

A demonstration deployment runs at `https://facial-reconstruction-atlas.vercel.app` containing **fully scrubbed data and generated placeholder images only**. An earlier demonstration deployment carrying the de-identified case metadata has been retired.

Vercel is not a project dependency and is not part of the production path. Nothing in the application requires it; `build.py` and `vercel.json` are the only Vercel-specific files and are excludable from a production deployment. The internal deployment does not depend on it in any way.

The production intent is hosting **inside the UVA Health environment**, reachable only from the institutional network or VPN, authenticating against Entra ID via SAML, and reading images from the existing departmental share rather than holding a copy.

12.1 Whether the public demonstration instance remains

Once internal hosting exists, the public instance serves no purpose for the project and can be retired. There is a separate question of whether it is worth keeping available as a demonstration of the application itself, running on the fabricated sample data. It holds no research records and would be entirely independent of the internal deployment.

This project team is not proposing that unilaterally. It is raised here so that it is a decision rather than an oversight, and the team will follow whatever the PI and Health IT prefer. If it were kept, the intent would be to make its status unmistakable — a visible notice that the data is fabricated and that the instance is not a UVA

Health service, and no institutional branding implying otherwise.

Two technical notes bear on it. The public instance cannot use Entra ID, since its users would be outside the institution, so it would need its own separate authentication and would share no identity infrastructure with the internal deployment. And its data would remain the twelve fabricated sample cases; there is no configuration under which it reads the departmental share.

Related to question 15 in the accompanying email.

Appendix A — Database schema

Case data

`patients` — one row per case

COLUMN	TYPE	NOTES
<code>id</code>	INTEGER	Primary key, autoincrement
<code>folder_name</code>	TEXT	Unique; maps to the image folder
<code>patient_id</code>	TEXT	Study identifier (<code>ENT-###</code>)
<code>location_raw</code>	TEXT	Original free-text defect location
<code>defect_size</code>	TEXT	
<code>full_thickness</code>	TEXT	
<code>method_of_repair</code>	TEXT	
<code>general_flap</code>	TEXT	
<code>specific_flap_description</code>	TEXT	
<code>graft</code>	TEXT	
<code>graft_donor_site</code>	TEXT	
<code>notes</code>	TEXT	Surgical notes

`patient_locations` — one row per defect site per case

COLUMN	TYPE	NOTES
<code>id</code>	INTEGER	Primary key, autoincrement
<code>patient_row_id</code>	INTEGER	Foreign key → <code>patients.id</code>
<code>region</code>	TEXT	
<code>sub_location</code>	TEXT	

Unique on (patient_row_id , region , sub_location).

images — one row per photograph

COLUMN	TYPE	NOTES
id	INTEGER	Primary key, autoincrement
patient_row_id	INTEGER	Foreign key → patients.id
filename	TEXT	Filename only; joined to the image root at request time
stage	TEXT	Reconstruction stage
sort_order	INTEGER	Display order within the case

Unique on (patient_row_id , filename).

Application data

TABLE	COLUMNS	NOTES
users	id , username , password_hash , created_at	username unique, case-insensitive. Expected to be reduced when SSO lands
sessions	token , user_id , created_at	token is the primary key; user_id → users.id
favorites	user_id , patient_row_id , created_at	Composite primary key on the first two columns
comments	id , patient_row_id , user_id , body , created_at	Foreign keys to patients and users

No stored procedures, triggers, or views.

Appendix B — Endpoint inventory

Case data

METHOD	PATH	PURPOSE	AUTH TODAY
GET	/api/anatomy	Anatomic diagram region definitions	Session
GET	/api/filters	Filter dropdown vocabularies	Session
GET	/api/search	Search and filter cases	Session
GET	/api/cases/{folder_name}	Single case detail	Session
GET	/api/image/{folder_name}/{filename}	Stream one image	Session

Authentication

METHOD	PATH	PURPOSE	AUTH TODAY
POST	/api/auth/register	Create local account	None
POST	/api/auth/login	Authenticate	None
POST	/api/auth/logout	End session	Session
GET	/api/auth/me	Current user	Session

User content

METHOD	PATH	PURPOSE	AUTH TODAY
POST	/api/cases/{folder_name}/favorite	Toggle favorite	Session
GET	/api/favorites	List favorites	Session
GET	/api/cases/{folder_name}/comments	List comments	Session
POST	/api/cases/{folder_name}/comments	Add comment	Session
DELETE	/api/comments/{comment_id}	Delete own comment	Session + ownership

Development tooling — to be excluded from production deployment

METHOD	PATH	PURPOSE	AUTH TODAY
GET	/region-editor	Anatomy region editor	404 unless enabled, then session
GET	/api/dev/face-regions	Read region definitions	404 unless enabled, then session
POST	/api/dev/face-regions	Write region definitions	404 unless enabled, then session

Static files

GET /, GET /script.js, GET /styles.css, GET /static/face-reference.jpg

Every endpoint serving case data or images requires a session. POST /api/auth/register and POST /api/auth/login are unauthenticated by necessity; the four static-file routes serve the application shell and must load before sign-in. Development routes return 404 unless ENTDATABASE_DEV_T00LS=1. Sections 10.3 and 10.4 record what has been closed and what remains.